

An Agentic Question-Answering System for Cross-Office Administrative Procedure Integration

Yao-Ming Pai

Student ID: 112703056

Instructor: Prof. Ming-Feng Tsai

Department of Computer Science, National Chengchi University

Abstract

Administrative regulations and forms at a university are typically scattered across dozens of office-specific web pages and PDF documents; answering a question such as "how do I apply for a leave of absence" requires assembling the complete procedure, the relevant form, and the office location and contact information for every signature field that form requires, information that, in our setting, spans seven offices across a nine-stop signature chain for a single form. Because the underlying documents in this class of application are often institutionally sensitive and cannot be sent to a closed-source API, we target a system that runs on a model deployable locally; we study it here on a corpus of public university regulations, as a prerequisite for applying the same architecture where documents cannot be shared. Conventional retrieval-augmented generation, built on dense embeddings and a reranker, struggles with this kind of query even under this constraint: a single response routinely took over ten minutes, and tended to reproduce regulatory text rather than the cross-office information the form actually required, while rebuilding the embedding index remained costly as the corpus grew. We present a question-answering system that instead relies on keyword search over the raw corpus and whole-page reading, rather than vector-based retrieval, with the language model itself deciding when to search and which pages to read, replacing the fixed retrieve-then-generate pipeline. In building this system, we found that the model's tool-calling behavior was itself unstable, frequently failing to invoke the tools a query required, a difficulty compounded by using a model small enough to deploy locally rather than a frontier system; we address this by drawing on and integrating several lines of agent-design research, namely chain-of-thought reasoning, ReAct-style action-observation loops, plan-and-solve decomposition, query routing, and self-evaluation, into a single multi-turn architecture. Along the way, we also observed that a sub-decision's reliability under model judgment tends to depend on whether an objectively checkable signal is available for it, rather than on how much prompt engineering is invested in it; resolving such decisions deterministically consistently improved reliability wherever we tested it. We evaluate the resulting system, without fine-tuning the underlying model, across several representative query types (complete procedures, single facts, form downloads, contact lookups, and cross-office compound queries), checking each response against manually-verified expected content such as named contacts, extension numbers, and form IDs, using a closed-source model as an upper-bound reference. As the paper's central empirical contribution, we additionally construct and manually verify five further queries, each isolating a distinct cross-office coordination pattern found in the underlying corpus (parallel dual-office approval, sequential handoff, multi-stage chained review, conditional routing by case type, and cross-institution coordination), and find that reliability varies sharply by coordination pattern rather than uniformly across the architecture, with one failure traced to a specific, generalizable retrieval bug and closed. Results show that the system reliably covers the required cross-office information, with traceable sources, on Gemma4:31b, a mid-sized, locally-deployed open-weight language model. Code is available at: <https://github.com/AlanPai777/nccu-academic-rag>.

1 Introduction

Retrieval-augmented generation has become the standard approach for grounding language models in external knowledge, typically by embedding a corpus into independent chunks and retrieving the most similar ones for a given query. This chunk-independent design, however, struggles when the information a query requires is spread across multiple documents rather than contained in any single passage: retrieval systems built this way have been shown to underperform on structured, multi-part sources such as financial tables (Subramanian et al., 2026), and rebuilding a dense index as a corpus grows remains computationally expensive. A more recent line of work instead lets the language model itself drive retrieval, deciding what to search for and when across multiple turns, rather than relying on a single similarity lookup (Singh et al., 2025); some of these agentic approaches abandon dense retrieval altogether in favor of lexical search or direct interaction with the raw corpus (Sen et al., 2026; Salemi et al., 2026).

Despite this shift toward agentic retrieval, it remains unclear how to make tool-calling behavior reliable on a locally-deployed model without fine-tuning it. Local deployment is often a hard requirement for this class of application, where institutional documents may be sensitive and cannot be sent to a closed-source API; but models that can be run locally are typically smaller than frontier systems, and strategy selection that works reliably at frontier scale has been found to fail on smaller ones (Zhou et al., 2026). We study this problem on a corpus of public university regulations rather than confidential records, precisely because doing so lets us validate, openly and reproducibly, whether a local model can complete this kind of multi-hop, cross-office retrieval at all, a prerequisite for applying the same architecture where the underlying documents cannot be shared. In our own setting, a fully agentic pipeline would locate the correct regulation page but repeatedly fail to follow through and retrieve the specific form it referenced: an answer would cite the leave-of-absence regulation correctly, yet omit the nine-station signature chain the form itself lists, because the model simply did not call the tool needed to read the form.

In this work, we present an agentic question-answering system for cross-office administrative procedures at a university. Rather than retrieving embedded chunks, the system reads whole pages (Jiang et al., 2024) and narrows its search through iterative reasoning over keyword matches rather than vector similarity (Wang et al., 2025), with the language model itself deciding when to search and which pages to read. To stabilize the model’s tool-calling behavior (the instability described above), we draw on and combine several lines of agent-design research: chain-of-thought reasoning (Wei et al., 2022), ReAct-style interleaved reasoning and acting (Yao et al., 2023), plan-and-solve decomposition (Wang et al., 2023), query classification and routing (Jeong et al., 2024), and self-evaluation (Asai et al., 2024), into a single multi-turn architecture. Along the way, we observed that a sub-decision’s reliability under model judgment tends to depend on whether an objectively checkable signal is available for it, rather than on how much prompt engineering is invested in it; resolving such decisions deterministically, outside model judgment, consistently improved reliability wherever we tested it, an observation consistent with a broader, concurrent discussion of agent harness design (Zhang, 2026), though we arrive at it independently, through repeated testing within a single deployed system rather than by adopting it from prior work.

We evaluate the resulting system, without fine-tuning the underlying model, across several representative query types (complete procedures, single-fact lookups, form downloads, contact lookups, and compound queries spanning two offices), using Gemma4:31b, a mid-sized, locally-deployed open-weight model, and a closed-source model as an upper-bound reference. The system produces answers that reliably cover the required cross-office information across these query types.

Our contributions are as follows:

- We identify chunk-independent retrieval as a poor fit for administrative procedures that require assembling information from multiple offices and forms, and replace it with an agentic retrieval design based on keyword search and whole-page reading rather than vector similarity.

- We construct and manually verify five additional queries, each isolating a distinct cross-office coordination pattern absent from the single procedure example otherwise anchoring our evaluation, and show that the architecture’s reliability is pattern-dependent: near-ceiling on parallel and sequential coordination, markedly weaker on cross-institution retrieval, where we trace one failure to its root cause and close it with a general, deterministic fix rather than a rule written for the specific case that surfaced it.
- We integrate several established techniques for stabilizing multi-turn tool-calling, namely chain-of-thought reasoning, ReAct-style action loops, plan-and-solve decomposition, query routing, and self-evaluation, into a single architecture, and evaluate it on a locally-deployed model without fine-tuning it.
- Along the way, we observe that a sub-decision’s reliability under LLM judgment tends to depend on whether an objectively checkable signal is available for it, rather than on how much prompt engineering is invested in it; resolving such decisions deterministically consistently improved reliability across four unrelated instances of this pattern within the deployed system, including one found through the cross-office evaluation above.

2 Related Work

Agentic and embedding-free retrieval. A growing body of work questions the chunk-and-embed paradigm on which most retrieval-augmented systems are built. LongRAG (Jiang et al., 2024) retrieves and reads long, coarse-grained units (whole documents or document groups) rather than short independent chunks, on the premise that fine-grained chunking discards context a reader can no longer recover on its own. ELITE (Wang et al., 2025) removes the embedding step entirely, instead narrowing a large corpus down to a small relevant subset through a sequence of iterative language-model reasoning steps over the raw text. A parallel line of applied work reports that lexical search alone (plain keyword or BM25 matching, with no dense component at all) is competitive with, or superior to, embedding-based retrieval on a range of practical benchmarks (Sen et al., 2026; Subramanian et al., 2026; Salemi et al., 2026; Wang et al., 2026), including specifically on the kind of structured, multi-part documents that motivate our own cross-office setting (Subramanian et al., 2026). More broadly, recent surveys describe a shift away from static retrieve-then-generate pipelines toward agentic designs in which the language model itself decides when and what to retrieve, over multiple turns (Singh et al., 2025). Our system follows this direction: like ELITE, it performs no embedding step; like LongRAG, it reads coarse units, entire pages, rather than fragments. Unlike either, retrieval and reading in our system are not a fixed multi-step procedure applied uniformly to every query, but individual tool calls that the language model itself issues and sequences over the course of an ordinary multi-turn agent loop.

Stabilizing multi-turn reasoning and tool use. Several established techniques address different aspects of getting a language model to reason and act reliably across multiple steps. Chain-of-thought prompting (Wei et al., 2022) shows that eliciting an explicit intermediate reasoning trace, rather than asking a model to produce a final answer directly, substantially improves performance on tasks that require multi-step inference. ReAct (Yao et al., 2023) extends this idea to tool use, interleaving reasoning traces with actions and their observations so that each decision about what to do next is conditioned on what a previous action actually returned, rather than planned blindly in advance. Plan-and-Solve prompting (Wang et al., 2023) instead separates planning from execution outright, first prompting the model to decompose a task into subtasks before carrying each one out, which the authors show reduces a specific failure mode of direct chain-of-thought prompting: steps skipped or executed out of order on multi-step problems. Our agent loop combines these ideas rather than choosing among them: an explicit planning pass over the incoming query precedes tool

use, in the spirit of Plan-and-Solve, the tool loop itself follows an interleaved reasoning-action-observation cycle in the spirit of ReAct, and individual reasoning steps within it are elicited chain-of-thought style.

Query routing and self-evaluation. Closest to our own work are methods that route a query to different retrieval or generation strategies, or that evaluate a system’s own output before returning it. Adaptive-RAG (Jeong et al., 2024) trains a lightweight classifier to predict query complexity and routes accordingly among no-retrieval, single-step, and iterative multi-step strategies, substantially improving efficiency on simple queries without sacrificing accuracy on complex ones, though this requires collecting labeled training data for the classifier and retraining it whenever the underlying strategies change. Lightweight Query Routing for Adaptive RAG (Bansal and Agarwal, 2026) reports that a small routing model can approximate this decision at a fraction of the training cost, but still requires a dedicated trained component external to the generator. Self-RAG (Asai et al., 2024) instead trains the generator model itself, via a fine-tuned critic and an expanded output vocabulary, to emit special reflection tokens that decide when to retrieve and that critique the relevance, support, and usefulness of its own output, collapsing routing and self-evaluation into a single fine-tuned model rather than a separate classifier. All three approaches route or self-evaluate through a trained decision-making component, which is a poor fit for a deployment where sensitive documents rule out sending data to a training service and where the underlying open-weight model is not being fine-tuned at all. Our system performs the same two functions (deciding what kind of query it is facing, and judging whether the resulting answer is acceptable) through a two-layer query classifier (deterministic keyword matching, with an LLM fallback only when no keyword rule applies) and a two-stage self-evaluation gate (a deterministic checklist first, with an LLM judgment invoked only if that checklist passes). Unlike Adaptive-RAG, Lightweight Query Routing, and Self-RAG, neither mechanism requires training or fine-tuning any model component, so both can be revised directly whenever a new query pattern or failure mode is observed.

Agent harness reliability. A separate, more recent line of work examines not what an agent decides, but how the harness around it (the tools, prompts, context, and control flow surrounding a language model) shapes the reliability of those decisions. CAAF (Zhang, 2026) argues that, for a fixed underlying model, the design of this harness is itself an asset worth engineering deliberately, rather than a fixed scaffold whose only job is to expose tools to the model. Chen et al. (2026) take this further, proposing a framework that traces an agent’s failed trajectories to diagnose which layer of the harness (tooling, context, lifecycle orchestration, among others) was responsible, and to generate a targeted repair, rather than adjusting the harness broadly based on final outcome alone. Industry reports on deploying agents in production describe a similar experience, arguing for treating determinism and model judgment as a spectrum to be allocated deliberately rather than a binary choice (deepset Team, 2025; Singh, 2026). Our own experience building the system is consistent with this broader picture, though at a much smaller scale: across three unrelated sub-decisions in the pipeline (whether a fetched page is relevant enough to keep, whether a page mentions a downloadable form, and whether it names a specific office), resolving the decision with a deterministic check, wherever an objectively verifiable signal was available for it, was consistently more reliable than leaving the same decision to the language model’s judgment, even after targeted prompt revisions. Unlike Chen et al. (2026)’s automated diagnosis-and-repair pipeline, we identified and applied this pattern by hand, one sub-decision at a time; we treat it as a secondary, practical observation arising from applying the design choices above within our own deployed system, rather than as a general claim independent of the literature it echoes.

3 Method

We organize this section around the sequence of design decisions the system embodies, each introduced to address a limitation the previous one leaves open.

3.1 Retrieval primitives: keyword search and whole-page reading

A query that requires assembling information from several offices and forms is a poor fit for retrieval built on independently embedded chunks: the information a query needs is not contained in any single passage, so a single similarity lookup has no way to know in advance which fragments, out of many scattered across unrelated pages, belong together. Our system does not embed the corpus into dense vectors at all; the corpus is instead indexed only for full-text search, using SQLite’s FTS5 extension, and the keyword-search tool ranks candidates with BM25 (Robertson et al., 1994) over that index, before a lightweight language-model judgment selects the best match among the top candidates it returns. Beyond this keyword-search tool, the system exposes retrieval as four further tools that the language model calls directly and as many times as it judges necessary, five tools in total: a literal substring search for exact strings such as form IDs, a tool that reads a given page in full, a tool that follows the hyperlinks a page contains, and a tool that fetches a specific form by its ID. Each of these returns a whole page or document rather than a fragment of one.

Everything a tool returns over the course of answering a query is kept, not discarded once used: each observation is appended to a running record that persists for as long as the query is being answered, so that a page read several tool calls ago is still available when the system eventually synthesizes an answer, not only the most recently fetched one. This is, in effect, a second route to the same goal LongRAG pursues (Jiang et al., 2024), giving the model more raw, unfragmented context to work with rather than a few short retrieved snippets, reached by accumulating several whole-page reads across one query’s tool calls rather than by retrieving one long unit at once. Left unbounded, this record would grow without limit across an entire multi-question conversation, and content fetched for an earlier, unrelated question could leak into a later one’s answer; between conversation turns we therefore collapse a completed turn’s full record down to just the question asked and the answer it produced, keeping only what a later question might actually need from it.

Which tool to call, how many times, and in what order is left entirely to the model, conditioned on everything accumulated so far, rather than fixed in advance. This freedom, however, does not by itself produce reliable multi-step behavior. Section 3.2 describes how we stabilize it.

3.2 A multi-turn loop for stable tool use

Given free choice over five tools and no fixed sequence in which to call them, a model can fail to invoke a tool a query actually requires, even when the tool exists and the information is retrievable: it locates the right regulation page, but never follows through to read the specific form that page references. We address this with a loop that structures, without removing, the model’s freedom to decide what to do next. Before each tool call, the model is first asked to state, in its own words, what it is now looking for, given everything read in the conversation so far, an explicit chain-of-thought-style reasoning step (Wei et al., 2022) that keeps the model’s current objective as a persistent, inspectable piece of state rather than leaving it implicit inside a single long generation. On the first turn only, the query is also routed to a specific subdomain of the corpus, narrowing what the tools search within before any tool call happens at all, an upfront planning pass, in the spirit of Plan-and-Solve (Wang et al., 2023), run once rather than interleaved with the steps that follow. The model then calls a tool, observes what it returns, restates what it is now looking for in light of that observation, and repeats, deciding for itself when it has gathered enough to stop; there is no fixed number of steps. This interleaving of an explicit restated objective, a tool call, and the observation that follows is a ReAct-style loop (Yao et al., 2023): it is what lets the second tool call, reading the form, actually get made, since each step is conditioned on what the previous one returned, rather than planned blindly before anything was retrieved.

Even inside this loop, however, certain sub-decisions recur across many different queries in a narrow, checkable form (whether a page just read is the specific form a query needs, or names a specific office), and

leaving these to the same open-ended judgment that governs the rest of the loop reintroduces the reliability problem this section was meant to solve. Section 3.3 describes how such decisions are instead resolved deterministically.

3.3 Deterministic routing for recurring sub-decisions

Two points in the system route execution deterministically rather than by model judgment, shown as the two shortcuts out of the ordinary loop in Figure 1. First, before any tool call happens, the incoming query itself is classified into one of four categories (a compound query, a resource request, a contact request, or an ordinary knowledge query) by a two-layer classifier: a set of deterministic keyword rules is checked first, and a language-model classification is invoked only when no rule matches. A query classified as a resource or contact request is routed directly to a dedicated resource-lookup or contact-lookup step, skipping the tool loop’s first pass entirely, since both kinds of query have a narrow, recognizable shape, such as "where do I download form X" or "what is office Y’s phone number," that an open-ended tool loop would otherwise have to rediscover by trial and error each time.

Second, this same routing is also checked continuously while the tool loop is running: every observation returned by a tool is matched against two fixed patterns, one for a form ID and one for an office name, regardless of what the model itself was doing at that step. A match routes execution to the resource-lookup or contact-lookup step deterministically; the model cannot override or decline this routing. We adopted this design after finding, across three unrelated instances of the same underlying shape of decision (whether a form was referenced, whether an office was named, and, earlier in development, whether a fetched page was relevant enough to keep) that a deterministic check was consistently more reliable than leaving the equivalent judgment to the model, wherever an objectively verifiable signal was available to check against. This is the same observation discussed in Section 2 in relation to agent harness design. The resource-lookup step also checks its own result against the office-name pattern before returning, since a fetched form’s own text often names the office where it must be submitted; when it does, execution chains directly into the contact-lookup step before continuing, rather than requiring the query to pass through the tool loop again just to notice what the form itself already said. Both the resource-lookup and contact-lookup steps, once finished, feed back into the ordinary loop described in Section 3.2 rather than ending the query outright, so that even a query resolved entirely through deterministic routing still gets a pass through the reactive loop afterward, in case more information is needed.

3.4 Compound queries: decomposition into independent sub-queries

A query that asks about two unrelated things at once, for instance a leave-of-absence procedure and a library’s phone number in the same message, fits none of the routes above well: the evidence for one half has nothing to do with the evidence for the other, and running both through a single loop risks one sub-question’s retrieved pages leaking into the other’s answer. When the initial classifier detects a compound query, it is instead split into independent sub-queries, each of which runs through its own separate instance of the loop from Section 3.2 before the results are combined for a final answer. This follows the general idea, explored by RQ-RAG (Chan et al., 2024) and, for rewriting a single query before retrieval, by Rewrite-Retrieve-Read (Ma et al., 2023), of decomposing or rewriting a query before searching rather than searching for it as given. Our split is a fixed, deterministic rule based on sentence-level punctuation rather than a learned rewriting step, which is adequate for the narrow case it targets (two or more administrative sub-questions concatenated into a single message) rather than the more open-ended query reformulation these methods are designed to handle.

3.5 Synthesis and a two-stage self-evaluation gate

Once the loop, the deterministic routing steps, or the compound-query decomposition has gathered what it judges to be enough evidence, a synthesis step generates the final answer from everything read so far. A query being routed correctly and every tool call succeeding does not, on its own, guarantee that this answer is complete: content the corpus actually contained can still be dropped in synthesis. We therefore run a self-evaluation step uniformly after every path through the system, not only the ones that took the long way through the loop. It is two-stage: a deterministic checklist first checks the answer against a small set of required-content rules at no model cost, and only if that checklist passes does a language-model judgment run, to catch failures the checklist has no fixed rule for. This resembles the self-evaluation mechanism in Self-RAG (Asai et al., 2024), discussed in Section 2, though ours is a single pass/fail gate evaluated once per attempt rather than a weighted, tunable combination of several critique signals scored per generated segment. If either stage rejects the answer, control returns to the rewriting step for another attempt rather than terminating immediately; we do not cap the number of retries directly, having found in an earlier version of the system that a fixed retry limit sometimes cut off an attempt that would otherwise have succeeded. The only remaining safeguard is a generous ceiling on the total number of turns, so that a query the system genuinely cannot answer still terminates eventually rather than retrying without end.

4 Experimental Setup

4.1 Query types and evaluation criteria

We evaluate the system on six representative query types, chosen to cover the shapes of question the architecture in Section 3 was built to handle: a complete multi-office procedure (leave of absence), a variant of the same procedure that tests whether the system generalizes rather than having simply memorized the first (resumption of study), a single factual lookup (course-credit limits), a resource request (downloading the leave-of-absence form), a contact lookup (a specific office’s extension), and a compound query combining two of the above into one message. Each query type is checked against its own gold-standard checklist of required content, hand-verified against the underlying regulations and forms, rather than a single rubric applied uniformly across all six: what counts as a complete answer differs by kind of query, since a procedure query must cover every office and signature step, a contact query only needs the right extension range and office name, and a form-download query is judged on whether the correct form is surfaced directly rather than on how much surrounding explanation accompanies it. Table 1 summarizes each type, the number of checklist criteria it carries, and what it specifically tests for.

Table 1 covers query *shape*, the different kinds of question a student might ask. A second, orthogonal axis is the specific cross-office coordination *structure* a correct answer must reconstruct, which is the paper’s central claim and an axis Table 1’s own procedure row tests only once, with a single seven-office example. We isolate this axis directly with five additional queries, each built around a form or procedure whose signature chain follows a structurally distinct coordination pattern absent from the corpus’s leave-of-absence example: two offices approving independently and in parallel; a two-step handoff where the second office’s action depends on the first office’s own signed output, not merely its approval; a three-stage chain where two departments and a central registrar each sign off in sequence; a single request whose reviewing office depends on which of several conditions applies; and a request that must be routed not just to the right office within the university but to the right partner *institution* among several with similar names. Table 2 summarizes these five.

Each checklist item across both tables is scored 0, 1, or 2: 2 if the answer contains at least a minimum number of that item’s required keywords, 1 if it contains some but fewer than that minimum, and 0 if it contains none, so a partially correct answer is credited proportionally rather than treated the same as a

Table 1: Query types and what each evaluates.

Query type	Example	Crit.	What it specifically tests
Procedure	how to apply for a leave of absence	13	Complete coverage of every office and signature step in a nine-stop chain across seven offices
Procedure variant	how to resume study after a leave of absence	9	Generalization to a second procedure rather than memorization of the first, including correctly withholding an approval step the first procedure requires but this one does not
Knowledge	the course-credit limit per semester	5	A single factual lookup with no multi-office structure
Resource	where to download the leave-of-absence form	3	Whether the correct form is surfaced directly, not how completely it is explained
Contact	the cashier’s office phone number	2	The right extension range and office name, deliberately not a specific staff member’s name, since staff turnover would otherwise make the criterion brittle
Compound	the leave-of-absence procedure and the library’s phone number, asked together	15	Whether both sub-answers remain complete rather than one diluting the other

missing one.

Each Table 1 checklist was constructed the same way: Claude Sonnet 4.6 was asked to answer the query directly (Section 4.2), and its answer was verified against the underlying regulations and forms (confirmed correct and complete, rather than trusted outright) before being converted into checklist items. The procedure checklist was additionally refined over the course of development, as specific gaps were discovered through failed runs and added as new criteria; the other five checklists have not been revised since their initial construction. Because every checklist derives from a verified Sonnet answer, Sonnet’s own answers are expected to satisfy them; the results in Section 5 accordingly report how closely Gemma4:31b, run through the architecture in Section 3, reaches this same verified standard, rather than treating Sonnet’s own pass as a separate finding. Table 2’s checklists were constructed directly from the underlying source documents instead, the relevant form’s own text and its signature-chain table, read in full, rather than from a verified model answer, since these five queries were built specifically to probe whether the system reconstructs a coordination structure visible in the source, independent of any one model’s phrasing of it. Three of Table 2’s five checklists were each revised once during evaluation, after an automated run’s score diverged from a manual reading of an already-correct answer; Section 5.2 reports each correction alongside the result it applies to, rather than silently folding it into the checklist beforehand.

A further limitation of this evaluation mechanism is worth disclosing directly: not every checklist item is checked by keyword matching. A handful of items per query type, such as whether the answer hallucinates content, or in one case, whether a procedure variant incorrectly requires an approval step the original procedure needs but the variant does not, carry no keywords at all and are scored automatically as passing, on the assumption that a human has verified the specific answer under evaluation. Running the evaluation script alone, without that manual step, can therefore overstate a result: in one of the runs reported in Section 5.1, an answer that fabricated an approval step the underlying form does not require was still scored as passing this specific check, because no one had verified that particular answer against the check’s intent. We treat the scores in Section 5.1 as having been through this manual check; a score obtained by running the script alone, without it, should not be taken at face value.

A related but opposite failure can also occur among items that do carry keywords: a keyword chosen to detect one fact can coincidentally overlap a substring used for an unrelated fact elsewhere in the same

Table 2: Cross-office coordination patterns and what each evaluates.

Pattern		Example	Crit.	What it specifically tests
Parallel office	dual-	which offices must approve a department transfer	6	Two offices signing off independently on the same request, neither depending on the other
Sequential off	hand-	how to get a dorm deposit refunded after moving out	6	A second office’s step requires the first office’s own signed output, not just its approval
Multi-stage chain		which offices must approve a double-major/minor application	6	Two departments and a central registrar signing off in sequence, one link longer than the parallel case
Conditional branching		which office reviews which category of exchange-credit course	7	The reviewing office is determined by a condition in the query (course type), not fixed
Cross-institution		which offices must approve a cross-institution double major	7	Correctly identifying which of several similarly-named external institutions the query refers to, not just which internal office

source document, producing a false pass rather than a false miss. We found and corrected exactly this in one of Table 2’s checklists (the conditional-branching pattern, Section 5.2): a credit-limit keyword collided with an unrelated deadline phrase sharing the same fraction, initially scoring an answer as passing a check it had in fact failed, since the answer never stated the credit limit at all. Two further Table 2 checklists needed the opposite correction, widening a keyword that was too narrow to recognize an already-verified-correct answer’s own valid phrasing (a job-title synonym, and a numeral written in Arabic rather than Chinese digits), the same kind of artifact already noted for the knowledge query in Section 5.1. Both directions of error are disclosed here precisely because they cut opposite ways: the false-pass direction can make the system look more reliable than it is, and the false-miss direction can make it look less reliable than it is; neither is caught by the human-verification step described in the preceding paragraph, which only guards against a checklist item passing with no keywords to check at all.

The query types were chosen to reflect real questions this deployed system is meant to answer for students, in a domain where correctness can be checked directly against the university’s actual regulations and forms, such as which office to visit, which extension to call, which form ID to retrieve, rather than against a more open-ended notion of what makes a "good" answer. This evaluation is correspondingly small in scale: each query type is represented by one exemplar query with a hand-built checklist rather than by a large benchmark of independently sourced examples, and queries were run once rather than repeated to measure variance across the language model’s sampling. We treat the results in Section 5 accordingly: as evidence that the system handles each targeted case correctly, checked against ground truth that can be directly verified, rather than as a statistically powered claim about the full space of administrative queries a real deployment would face.

4.2 Models compared

We run the full system described in Section 3 on Gemma4:31b (Gemma Team, 2026), an open-weight model small enough to deploy locally, without fine-tuning it, consistent with the motivation in the introduction that the architecture must work on a model that can be run without sending documents to an external API. As an

upper-bound reference, we also obtained answers to the same queries using Claude Sonnet 4.6 (Anthropic, 2026) directly, operating as a general-purpose coding agent with access to the same corpus, rather than through the pipeline described in Section 3: its own native tool use and reasoning stand in for the architecture proposed here, rather than being constrained by it. This comparison is not intended as a fair benchmark between two systems of comparable cost, deployability, or even architecture, but as a check on how much headroom the problem itself leaves on the table when a capable model is given the same corpus and freed from the specific design choices in Section 3.

4.3 Corpus and implementation

The underlying corpus was crawled from 153 subdomains of the university’s web presence, yielding 226,795 successfully fetched HTML pages and documents (approximately 35.5 GB of raw content). After extraction, the searchable corpus that every retrieval tool in Section 3.1 operates over directly is a set of per-page text records, one JSON line per page, totaling 203,924 lines across all subdomains, roughly 900 MB of plain text; not every fetched file survives this step, since the extraction pipeline only attempts HTML and PDF files, and a small remainder of those return no usable text. Extraction itself combines several methods depending on document type: HTML pages are parsed for their main content region, with office contact details specifically pulled out of page footers before the rest of the footer is discarded, and in-line links preserved as markdown; PDF pages are extracted with a layout-aware library that converts detected tables to Markdown to keep column structure intact, falling back to plain-text extraction otherwise and to optical character recognition for scanned or otherwise unrecoverable pages. A separate index built during crawling maps each form’s ID to its download URL and metadata across all 153 subdomains, directly backing the form-fetching tool from Section 3.1; supplementary content fetched through this same mechanism for 59 of the subdomains is folded into the searchable corpus alongside the ordinary extracted pages.

This is the same public corpus that motivated the local-deployment requirement described in the introduction, used here as a validation testbed for an architecture ultimately intended for documents that cannot be shared.

For evaluation, each query is matched to its gold-standard checklist by the exact text of the query itself, rather than by the category the system’s own classifier (Section 3.3) assigns it at runtime. This keeps the two measurements separate: whether the classifier routes a query correctly is a property of the routing mechanism, and conflating it with whether the final answer is correct would make a routing error and an answer error indistinguishable in the resulting score.

5 Results and Analysis

5.1 Main Results

Table 3 summarizes the outcome for each of the six query types on Gemma4:31b, checked against the verified checklists described in Section 4.1 and, where a checklist item required it, manually confirmed against the answer text rather than taken from the automated score alone. The procedure query reaches 24 of 26 points: every structural requirement is met, including the nine-stop signature chain across seven offices, the conditional stops, the three-tier approval ending in the dean’s office, the correct contact for each office, the pickup method, and the mental-health disclosure, but the answer omits the specific refund percentage table, present only on a linked page the agent did not follow within its three tool calls, and cites one of two extensions that genuinely appear for the cashier’s office across different pages of the corpus, rather than the specific one the checklist happens to have been built around.

The procedure variant reaches 16 of 18 points after manual verification, lower than the automated script’s own report of 18: the automated score credits this query’s dedicated check for a specific error, fabricating an

Table 3: Result by query type (Gemma4:31b), manually verified against the checklists in Section 4.1.

Query type	Result
Procedure	24/26 (refund table incomplete; one contact number reflects a genuine two-version discrepancy in the source corpus)
Procedure variant	16/18 (fabricates a dean’s-office approval step an early-resumption form does not require)
Knowledge	8/10 (content fully correct; the miss is a keyword-phrasing artifact, not a factual gap)
Resource	2/6 (answered with a full procedure explanation rather than a direct link; download link host unverified)
Contact	4/4 (reports both extensions that appear for the cashier’s office rather than picking one)
Compound	25/30 (see Section 5.4)

approval step the early-resumption procedure does not require, without actually checking for it, since that item carries no keywords and is scored as passing by default. Verified against the underlying form directly, the answer does commit this exact error, adding a dean’s-office approval step to the early-resumption chain that the form itself does not list; the corrected score reflects this.

The knowledge query’s answer is fully correct in substance: every required figure, including the semester credit floor for undergraduates, is present, but the automated score of 8 out of 10 undercounts it, because the answer states the floor as part of a range ("12 to 25 credits") rather than as an isolated phrase the keyword check was written to expect. The resource query is the weakest result, and the specific way it fails is informative: 休學申請表在哪裡下載 is exactly the kind of query the deterministic classifier in Section 3.3 is meant to route directly to the resource-lookup step, skipping the loop entirely, yet this run took eight turns and returned a full procedure explanation instead of a direct link: the query evidently fell through to the ordinary loop rather than being classified as a resource request. This is consistent with the tool-calling and classification instability on non-frontier models discussed in the introduction (Zhou et al., 2026): the same instability that motivates the architecture in Section 3 in the first place can, on a given run, affect the classifier’s own decision at its entry point. The download link this run returned also pointed to a different host than the one the form is normally served from, whose validity we have not independently confirmed; manually scored, only the form ID it identified is verifiably correct, giving 2 of 6 points. The contact query is fully correct, again reporting both extensions that appear for the cashier’s office across the corpus rather than picking one.

The compound query is discussed in detail in Section 5.4. Section 5.2 extends this evaluation with five further queries built specifically around cross-office coordination structure, the axis Table 2 isolates.

5.2 Cross-office architecture stress test

Section 5.1’s procedure query is the only point in the preceding evaluation where a single answer must reconstruct information spanning more than two offices, and the only query type built around cross-office coordination specifically, as opposed to query shape more broadly (Section 4.1). To evaluate the paper’s central claim on more than this one example, we ran the five queries in Table 2, isolating one structurally distinct cross-office coordination pattern per query, each checked against a checklist built directly from the relevant form’s own text (Section 4.1) and manually verified against the system’s actual output rather than assumed correct from an automated score alone.

The parallel dual-office pattern (which offices must approve a department transfer) scores 12 of 12: the answer correctly names both the originating and receiving department as independent approvers, the registrar’s own two-tier internal sign-off, the application window, and the one-transfer-per-degree limit, with no items requiring correction after manual review.

The sequential-handoff pattern (dorm deposit refund) scores 12 of 12 after one correction to the checklist itself: the system’s answer, checked against the form’s own text, correctly names the room-inspection step, the three-day handoff window to the housing office, the required documentation, and the international-student exception, but the automated score initially undercounted two of these because the answer used phrasing present in the form’s own text (a job-title term for the inspecting staff, and an Arabic rather than Chinese numeral for the three-day window) that the checklist had not anticipated; both corrections widen the checklist’s accepted phrasing rather than change what content the answer must contain.

The multi-stage chain pattern (double-major/minor application) scores 10 of 12: one checklist item required the same kind of phrasing correction as above, but a second item, requiring the answer to state that an approved external major automatically forfeits an existing one, remains unmet after correction. Manual inspection of the system’s tool-call record could not initially determine whether the underlying form content was ever retrieved and then dropped during synthesis, or never retrieved at all, because the trace used for this inspection filters to only two message types (tool invocations and their results) and omits a third type the retrieval step uses specifically to inject fetched form content into the running conversation; turn-count evidence in the same log is consistent with the form having been retrieved, which would make this a synthesis-level omission rather than a retrieval-level one, but the trace itself cannot confirm this directly, a concrete instance of the observability gap noted in Limitations.

The conditional-branching pattern (which office reviews which category of exchange-credit course) scores 12 of 14 after one checklist correction in the opposite direction: an item intended to check for a stated credit-limit fraction initially scored as passing because its keyword happened to also match an unrelated deadline phrase sharing the same fraction, not because the answer actually stated the credit limit; corrected, the item fails, and the 2-point gap is genuine, the system correctly routes each course category to its reviewing office (physical education, general-education, and foreign-language offices, respectively) but never states the credit-limit rule itself.

The cross-institution pattern (which offices must approve a cross-institution double-major application) is the weakest result and the most informative failure in this evaluation. On the first attempt, the system scored 3 of 14: rather than the four-university consortium the query named, it retrieved and confidently elaborated on an entirely different bilateral agreement between the university and an unrelated arts institution whose name shares no substring with the query’s own, reproducing that agreement’s real contacts, deadlines, and procedure faithfully (no hallucinated content, by the same standard applied in Section 4.1), but answering a different, unrelated question than the one asked, and on regenerating the answer a second time from the same query, going further and asserting a false subordinate relationship between the two institutions outright. Tracing the failure to its source located it precisely: the retrieval judge’s view of each candidate page included the anchor text of a markdown link pointing elsewhere on the page, and one candidate’s anchor text happened to name the consortium the query asked about even though the page’s own body concerned the unrelated institution; the judge had no way, from the text it was shown, to distinguish a link’s destination from the page’s own content. We corrected this with a general, deterministic fix, stripping link spans before the judge sees a candidate, described fully as a fourth case in Section 5.3, rather than a rule written for this specific pair of institutions. Re-run after the fix, the same query scores 9 of 14, correctly identifying the consortium and its two-tier per-institution approval structure; the remaining gap is a set of narrower omissions (the consortium’s member-institution names, and its enrollment quota) consistent in kind with the multi-stage chain pattern’s own residual gap above, not a recurrence of the wrong-institution failure the fix targeted.

Across all five patterns, what most closely determines reliability is not query shape (Section 4.1) but coordination structure: parallel and sequential two-office patterns reach ceiling immediately, a three-stage chain and a conditional branch each leave one narrow, genuine content gap, and cross-institution coordination was the only pattern where the system failed at the level of *which* institution the query concerned, rather than at the level of *which details* about the correct one to include, the distinction the following subsection

Table 4: Result by cross-office pattern (Gemma4:31b), manually verified against the checklists in Table 2.

Pattern	Result
Parallel dual-office	12/12
Sequential hand-off	12/12 (checklist corrected: form used a job-title synonym and an Arabic numeral the original keywords did not anticipate)
Multi-stage chain	10/12 (a forfeiture rule from the form is missing from the answer; not distinguishable from the trace alone whether never retrieved or dropped in synthesis)
Conditional branching	12/14 (checklist corrected for a keyword collision; the credit-limit rule is genuinely absent from the answer)
Cross-institution	3/14 → 9/14 after fixing the retrieval judge (Section 5.3); remaining gap is member-institution names and an enrollment quota, not the original wrong-institution failure

examines directly.

5.3 Verifying the deterministic-routing design

Section 3.3 described three points in the pipeline where a decision that could have been left to the language model’s own judgment is instead resolved deterministically. Each was adopted after directly comparing it against the model-judgment alternative it replaced, rather than by design intuition alone.

Triggering the resource-lookup step. Earlier in development, whether to fetch a form referenced mid-conversation was left to the agent’s own decision about which tool to call next. The agent frequently reasoned correctly that a form was referenced, its own restated objective would mention the form ID, but did not reliably follow through and call the tool that would fetch it, producing an answer that cited the correct regulation but omitted the form’s own content. Replacing this with a deterministic check, a fixed pattern matched against every tool observation, rather than a decision left to the agent, removed this specific failure mode; the agent no longer needs to remember to fetch a form it has already, in effect, decided is relevant.

Narrowing candidates before judging them. An earlier version of the resource-lookup step judged which of roughly 32 candidate forms were relevant to fetch, using all of them at once. A plausible-seeming optimization narrowed this to just the one or two candidates whose IDs were directly detected in the retrieved text, on the reasoning that fewer, more targeted candidates should make the judgment more accurate; a small sample of runs (three to five) appeared to confirm this. A same-timestamp, matched-input comparison across eight trials each showed the opposite: the original version, judging over the full candidate list, succeeded on all eight trials, while the narrowed version succeeded on none. Investigating why led to a different, more fundamental fix: an earlier, stable version of the pipeline turned out to have no pre-fetch relevance judgment at all, it fetched every detected form unconditionally and left the question of which one to describe in the final answer entirely to the synthesis step, where a mistaken filter only produces a slightly cluttered answer rather than silently dropping data with no way to recover it. Removing the pre-fetch judgment step entirely, deferring relevance filtering to synthesis, is the version now in place.

Detecting office names. Deciding whether a passage of retrieved text names a specific office was originally left entirely to a language-model judgment call. Across a fixed test set, this judgment succeeded on 14 of 15 cases. Adding a deterministic substring pre-check, testing the text directly against a list of known office names before falling back to model judgment only when the pre-check found nothing, brought the same test set to 10 of 10 correct, the full set attempted after the change.

Judging candidate relevance without link-anchor noise. The cross-institution failure in Section 5.2 traced to a fourth instance of the same underlying problem, this time in the retrieval judge itself: the 200-character excerpt shown to the judge for each candidate page included, unmodified, the anchor text of any markdown link the page contained near its start, so a link’s description of the page it pointed to could be read as if it were the current page’s own content. Our first attempt at a fix added an instruction to the judge’s prompt asking it to verify that a query’s named institution matched the candidate’s own content rather than a link label; this was deliberately rejected in favor of a fix that did not depend on the judge correctly applying a new, query-shape-specific rule every time. The fix instead strips every markdown link span, both its anchor text and its destination, from the excerpt before the judge sees it, a data-layer correction available for any future candidate with the same shape of noise rather than a rule scoped to this one pair of institutions. We verified it against both the failing case it targeted and an unrelated, already-passing regression case (the procedure query from Section 5.1, re-run after the fix): the cross-institution query improved from 3 to 9 of 14 (Section 5.2), while the procedure query scored 25 of 26, unchanged within the range already reported.

These four cases span four unrelated parts of the pipeline (tool-calling follow-through, a relevance filter, an entity-detection step, and a candidate-relevance judgment) but share the same underlying pattern: in each, an objectively checkable signal was available, and checking it directly outperformed asking the model to judge it, however the prompt asking for that judgment was worded, and in the fourth case, worded differently. This is consistent with the harness-reliability literature discussed in Section 2 (Zhang, 2026; Chen et al., 2026), though here the pattern emerged from repeated testing within a single deployed system rather than from adopting a proposal already established there.

5.4 Case study: a compound query

To illustrate why the decomposition step in Section 3.4 exists, we walk through 如何辦理休學，圖書館的電話是多少 ("how do I apply for a leave of absence, and what is the library’s phone number"), the compound query introduced there.

Before decomposition existed, an earlier version of the pipeline handled a compound query with a single synthesis call spanning both sub-topics. On this exact query, that version answered the library half correctly in full, but the leave-of-absence half, checked against the same checklist used for the procedure query on its own, dropped from 25 of 26 points, achieved when asked alone at the time, to 14 of 26 asked as half of this compound query. Routing was not at fault: content for both topics reached the synthesis step. The failure was dilution: a single call handling two topics at once produced an answer where entire categories of required content for one of them disappeared, including the form to download and four of the procedure’s seven office stops.

The current system avoids this specific failure by construction: the query classifier (Section 3.3) routes a compound query to the decomposition step, which splits it into independent sub-queries before either reaches a tool or a synthesis call, and each sub-query runs through its own separate instance of the loop from Section 3.2 and reaches its own synthesis before the two answers are combined. Run today, the procedure half of this query scores essentially the same as it does asked alone (Section 5.1): the severe dilution the earlier version exhibited, whole categories of content vanishing, does not recur, though a milder version persists in one item: the mental-health disclosure, present when the procedure query is asked alone, is only partially present here, mentioning a pregnancy-related condition but not the mental-health center itself or its phone number.

The library half of the current answer, however, contains a different kind of error, unrelated to dilution: it names a phone extension for "圖書資訊與檔案學研究所," an academic department whose name shares the words "library" and "information," rather than "圖書館," the university library itself; these are two distinct offices the retrieved content did not clearly distinguish. This is not the failure decomposition was built to prevent; it is the same kind of office-name ambiguity discussed in Section 3.3 and Section 5.3, surfacing

here on a sub-query that did not go through the same deterministic office-detection check contact-classified queries receive directly. Decomposition solved the problem it targeted, the leave-of-absence answer is no longer hollowed out by having to share a synthesis call with an unrelated topic, without solving a second, independent problem that happens to appear in the same query.

Across the four preceding subsections, the results are broadly consistent with the pattern discussed in Sections 3.3 and 5.3: where a decision reduces to a deterministic check, the outcome is reliable: the contact query, resolved through explicit office-name matching, is fully correct; the compound query’s leave-of-absence half, no longer sharing a synthesis call with an unrelated topic once decomposition has run, is not diluted the way it was before that fix existed; and the cross-institution pattern in Section 5.2, the clearest single instance of this in the whole evaluation, moves from the weakest result in either table to a middling one once the corresponding judgment is resolved deterministically rather than left to the model. Where a decision still depends on model judgment without a comparable deterministic backstop, the same category of instability motivating this paper’s central design choice (Section 1) resurfaces: the resource query’s mis-routing, the compound query’s confusion between two similarly named offices, and the remaining, narrower gaps in Section 5.2’s multi-stage-chain and conditional-branching patterns are all decisions that, unlike the checks covered by Section 3.3 and Section 5.3, are not yet covered by one. The remaining gaps in this evaluation are, in this sense, informative about where to extend the same design principle next, rather than a departure from it.

6 Conclusion

In this paper, we present an agentic question-answering system for cross-office administrative procedures at a university, built around keyword search and whole-page reading rather than dense retrieval, an agent loop that combines established techniques for stabilizing multi-turn tool use, and deterministic routing wherever an objectively checkable signal is available for a decision that would otherwise be left to the language model’s own judgment. Evaluated on Gemma4:31b, a mid-sized, locally-deployable open-weight model, without fine-tuning it, the system reliably assembles complete, cross-office information, procedures, forms, and contacts, for the query types it was built to handle, and the specific points where it falls short are, as discussed in Limitations, largely traceable to gaps the same design principle has not yet been extended to cover, rather than to a failure of the principle itself.

7 Limitations

The evaluation in Section 4 and Section 5 is deliberately narrow, as already noted in Section 4.1: eleven query types across Table 1 and Table 2, each represented by one exemplar and, for all but one, a single run, is enough to check whether the architecture in Section 3 handles the cases it was built for, but not enough to establish how often it does so under the sampling variance inherent to a non-deterministic language model: a query answered correctly once in this evaluation may not be answered identically on a second attempt, and we have not measured that variance systematically. The one exception offers a narrow, informal data point rather than a substitute for that measurement: the cross-institution query (Section 5.2) was run twice before its fix, returning the same wrong institution both times, which is consistent with a structural retrieval bug rather than sampling noise, but a single repeated pair is far short of what would be needed to characterize variance across the evaluation as a whole. A related limitation lies in the evaluation mechanism itself: several checklist items, including the one specifically designed to catch a fabricated approval step in Section 5.1, carry no keywords and are scored automatically as passing unless a human separately verifies the answer under evaluation; without that manual step, the scores reported in Table 3 would have overstated the system’s reliability on at least one query. Future work could repeat each query multiple times to measure

sampling variance directly, and could replace the checklist items that currently rely on manual verification with automated checks specific enough to catch the same errors a human reviewer would.

A further limitation concerns observability during evaluation itself, distinct from the evaluation mechanism discussed above: the trace used to inspect why a given answer succeeded or failed renders only two of the several message types the pipeline’s internal state actually contains, omitting exactly the type used to inject retrieved form and office content into the running conversation. This made one specific diagnosis in Section 5.2 (the multi-stage-chain pattern’s missing forfeiture rule) inconclusive from the trace alone, leaving open whether the content was never retrieved or retrieved and then dropped during synthesis, two failures with different fixes. Extending the trace to surface this message type, even abbreviated to a marker line rather than its full content, would let future evaluation distinguish these two failure classes directly rather than by indirect turn-count inference.

A further limitation follows directly from the retrieval design in Section 3.1: keyword search and literal substring matching both fail in the same way when a query and the page that answers it use different words for the same thing: 休學 ("leave of absence") and 保留學籍 ("retaining student status") refer to closely related procedures but share no common substring. When this happens, the agent has no recourse but to retry with different keywords turn after turn until one happens to match. This is a cost problem as much as a correctness one: each unsuccessful retry is a full additional tool call and, typically, an additional language-model turn, on top of whatever the query would have needed regardless, so a query that should resolve in two or three turns can instead require many more simply because the first several keyword guesses missed. Future work could reduce how often this retry loop is needed by generating a short hypothetical answer to the query first and using its vocabulary, rather than the query’s own, to search: a lightweight variant of HyDE (Gao et al., 2023) that requires no change to the existing FTS5 index, since the hypothetical answer is used to expand the keyword query rather than to build a new embedding index. When repeated retries remain necessary regardless, moving from SQLite’s FTS5 to a keyword-search engine built for higher query volume, such as Lucene or Pyserini, would not close the vocabulary gap itself but would reduce the cost of each retry attempt.

Two of the specific errors surfaced in Section 5 point to concrete gaps in the architecture rather than in the evaluation. First, the resource query’s misrouting (Section 5.1) suggests that the deterministic classifier in Section 3.3, while effective where it fires, does not yet cover every case its keyword rules are meant to catch: its category definitions were written around specific query shapes (an office’s contact details, for instance) and can silently miss a related but distinct shape (a role-to-person lookup such as "who is the dean") without ever misclassifying anything the taxonomy actually defines. A promising direction, motivated by Adaptive-RAG’s finding that a small classifier trained on the model’s own demonstrated performance outperforms hand-written rules (Jeong et al., 2024), and by evidence that lexical features alone are sufficient for this kind of query-type classification without semantic embeddings (Bansal and Agarwal, 2026), would be to train a lightweight classifier, reusing the tokenization already built for keyword search (Section 3.1), as the primary router, escalating to the language model only when the classifier’s own confidence is low, with training labels drawn automatically from which path each historical query’s self-evaluation gate (Section 3.5) actually passed. Confidence-gated escalation of this kind is not itself part of either cited method, since both make a single deterministic classification at inference without a fallback tier, so this would be an extension building on their findings rather than a direct application of them. Second, the office-name confusion between the university library and a similarly named academic department (Section 5.4) shows that the deterministic office-detection check discussed in Section 3.3 and Section 5.3 does not yet run on every path that might need it: it was built for, and tested on, queries routed directly to the contact-lookup step, not sub-queries that reach an office name through the ordinary search loop. Extending the same deterministic check to cover this second path is a direct, low-risk way for future work to close this specific gap, consistent with the pattern of results in Section 5.3. Both of these remain open; a fourth, closely related gap of the same general shape, the retrieval judge conflating a candidate page’s own content with an unrelated link’s anchor text (Section

5.3), was found and closed within this same round of evaluation, which is offered here as one data point in favor of the same fix pattern rather than as evidence the two gaps above are already resolved.

A separate limitation lies in the corpus itself rather than in the system built on top of it: the two-extension discrepancy for the cashier’s office (Section 5.1) reflects genuinely inconsistent source data, which no retrieval or synthesis design can resolve without either curating the corpus directly or deciding, as a matter of policy, how the system should behave when its sources disagree, a decision we leave unresolved here, treating an honest report of the disagreement as preferable to an arbitrary choice between two real answers. Future work could instead resolve this during preprocessing, by periodically cross-checking office contact information across pages and flagging or reconciling discrepancies before they reach the corpus this system searches, rather than leaving each query to surface them individually.

A final limitation concerns readiness for the deployment this paper’s motivation ultimately points toward. The introduction motivates local deployment by the eventual goal of applying this architecture to institutionally sensitive documents, but the system as evaluated here has not been hardened for that setting: retrieved page content is passed into the synthesis prompt without being structurally separated from system instructions, leaving it open to indirect prompt injection from a compromised or maliciously edited source page; and the conversational state used across turns is held only in process memory, tied to a session identifier the server does not itself authenticate, so a guessed or intercepted identifier could expose another user’s conversation. Neither gap is specific to the architecture in Section 3, both are standard production-hardening concerns for any deployed LLM agent, but leaving them unaddressed means the local-deployment motivation in the introduction is not yet fully realized: future work should structurally isolate retrieved content from instructions before it reaches synthesis, and bind session state to an authenticated identity, before this architecture is applied to documents it cannot afford to expose.

References

- Anthropic. System card: Claude sonnet 4.6. <https://anthropic.com/claude-sonnet-4-6-system-card>, 2026.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*, 2024.
- Prateek Bansal and Saurabh Agarwal. Lightweight query routing for adaptive RAG: A baseline study on RAGRouter-Bench, 2026.
- Chi-Min Chan, Chunpu Xu, Ruibin Yuan, Hongyin Luo, Wei Xue, Yike Guo, and Jie Fu. RQ-RAG: Learning to refine queries for retrieval augmented generation, 2024.
- Ming Chen, Jing Wang, Zhi Liu, Yue Wang, Hao Zheng, and Qi Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws, 2026.
- deepset Team. AI agents and deterministic workflows: A spectrum, not a binary choice. <https://deepset.ai/blog/ai-agents-and-deterministic-workflows-a-spectrum>, 2025.
- Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *ACL*, 2023.
- Gemma Team. Gemma 4 technical report, 2026.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *NAACL*, 2024.

- Ziyan Jiang, Xueguang Ma, and Wenhui Chen. LongRAG: Enhancing retrieval-augmented generation with long-context LLMs, 2024.
- Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting for retrieval-augmented large language models. In *EMNLP*, 2023.
- Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. Okapi at TREC-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*, pages 109–126. NIST, 1994.
- Alireza Salemi, Chuan Zeng, Anish Nijasure, Jia-Huei Chung, Razieh Rahimi, Fernando Diaz, and Hamed Zamani. GrepSeek: Training search agents for direct corpus interaction, 2026.
- Sourav Sen, Aditya Kasturi, Eric Lumer, Anmol Gulati, and Vamsi K. Subbiah. Is grep all you need? how agent harnesses reshape agentic search, 2026.
- Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaei Khoei, and Athanasios V. Vasilakos. Agentic retrieval-augmented generation: A survey on agentic RAG, 2025.
- Harpreet Singh. Deterministic in the agentic era. <https://fractal.ai/article/deterministic-in-the-agentic-era>, 2026.
- Shiv Subramanian, Adewale Akinfaderin, Yu Zhang, Ishan Singh, Manasi Khanuja, Sanjay Singh, and Mariana Ladeira Tanke. Keyword search is all you need: Achieving RAG-level performance without vector databases using agentic tool use, 2026.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *ACL*, 2023.
- Peng Wang, Bo Xu, Shuo Wang, Ming Du, Xin Zeng, Hao Wu, Lei Zhang, and Li Zhang. BM25 wins at scale: A scaling study of retrieval-augmented generation paradigms, 2026.
- Zhen Wang, Shuo Gao, Rui Zhou, Hao Wang, and Lei Ning. ELITE: Embedding-less retrieval with iterative text exploration, 2025.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- Tao Zhang. Harness as an asset: Enforcing determinism via the convergent AI agent framework (CAAF), 2026.
- Hao Zhou, Zhixin Tan, Zheng Zhang, Yifan Fan, Yue Lin, Le Kang, Xin Song, Rui Li, Shuo Huang, Ang Yu, Yue Fan, Yifan Chen, Kai Xu, Xiao Liu, Yue Qin, Philip Torr, Chi Zhang, and Zheng Yin. Select-then-solve: Paradigm routing as inference-time optimization for LLM agents, 2026.

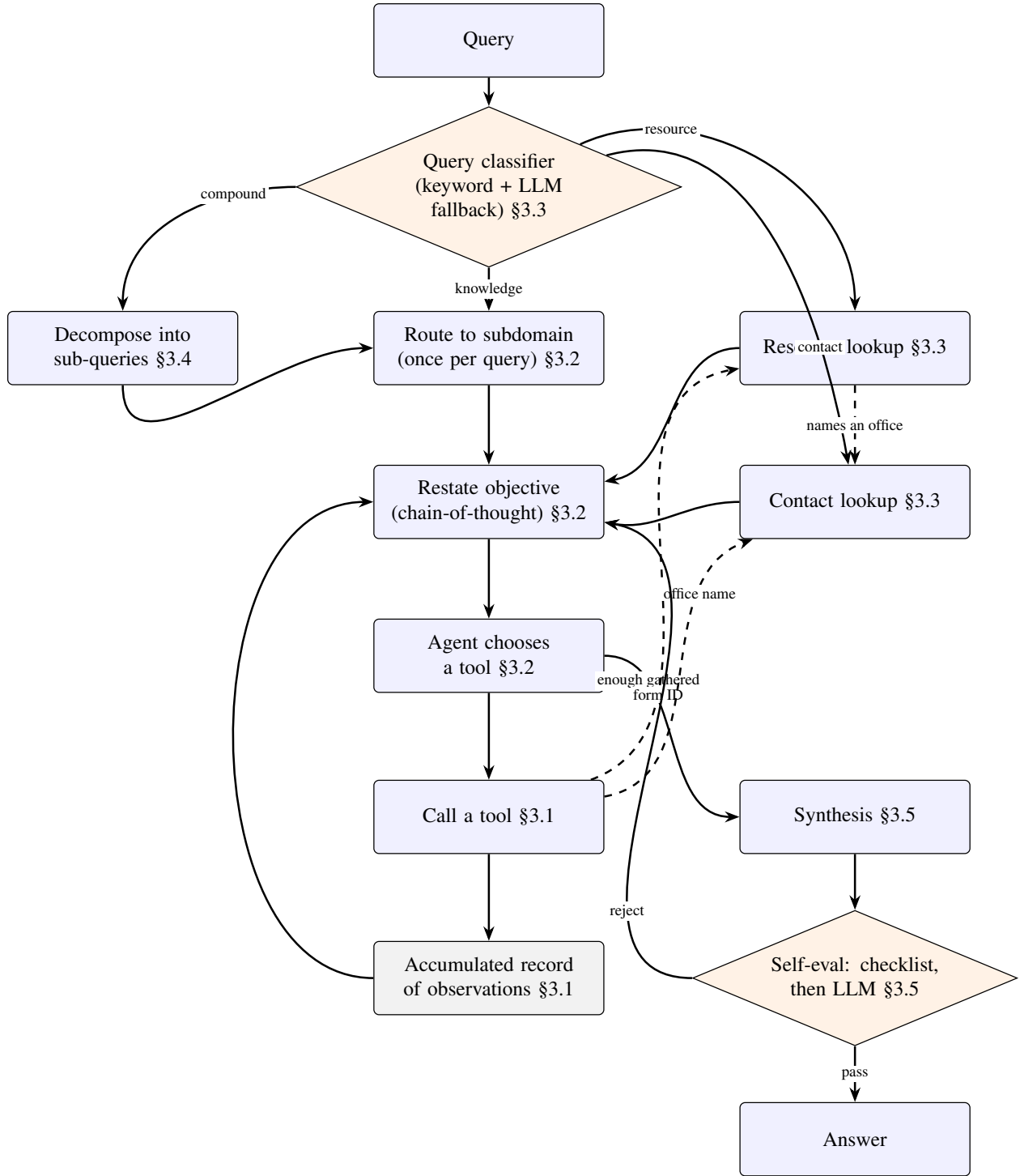


Figure 1: The system’s control flow. A deterministic classifier routes each query to a resource lookup, a contact lookup, a compound-query decomposition, or the ordinary multi-turn loop; within the loop, the same two deterministic checks can redirect execution to a resource or contact lookup mid-turn, and a resource lookup can chain into a contact lookup when the content it retrieves itself names an office. Every path converges on synthesis and a two-stage self-evaluation gate.